

Dijkstra's Algorithm

Dijkstra's algorithm is a greedy algorithm used to find the shortest paths from a single source vertex to all other vertices in a directed, weighted graph where all edge weights are non-negative.

Goal

Given a directed, weighted graph $G = (V, E)$ with non-negative edge weights and a source vertex s , compute the shortest paths from s to all other vertices in G .

Algorithm Idea

Dijkstra's algorithm maintains an estimate $v.d$ for the length of the shortest known path from s to each vertex v . Initially, $s.d = 0$ and all other $v.d = \infty$. The algorithm iteratively improves these estimates by "relaxing" edges.

It grows a "shortest path tree" T from the source s . In each step, it adds the vertex u with the smallest current distance estimate $u.d$ that is not yet in T to the tree. Once u is added, all its outgoing edges are relaxed.

Initialization

1. Set the distance estimate for the source vertex s to 0: $s.d = 0$.
2. Set the distance estimate for all other vertices $v \in V - \{s\}$ to infinity: $v.d = \infty$.
3. Initialize the predecessor of all vertices to **null**: $v.\pi = \text{null}$.
4. Maintain a priority queue P containing all vertices, keyed by their distance estimates $v.d$.

Steps

1. Extract the vertex u with the minimum distance estimate from the priority queue P .
2. Add u to the set of vertices for which the shortest path has been finalized (effectively, adding it to the shortest path tree T).
3. For each neighbor v of u (i.e., for each edge (u, v)):
 - **Relax** the edge (u, v) : If $v.d > u.d + w(u, v)$, update $v.d = u.d + w(u, v)$ and set $v.\pi = u$. This update also requires a **DECREASE-KEY** operation on v in the priority queue.
4. Repeat until the priority queue P is empty.

Pseudocode

```
DIJKSTRA( $G, s$ )  
  for each vertex  $v$  in  $V$ :  
     $v.d = \text{infinity}$ 
```

```

    v. = null
    s.d = 0

P = priority queue containing all vertices in V, keyed by v.d
// A common way is to insert all with infinity, then DECREASE-KEY(P, s, 0).

while P is not empty:
    u = EXTRACT-MIN(P) // Get vertex with smallest distance estimate
    for each edge (u, v) in Adj[u]: // For all neighbors v of u
        RELAX(u, v)

RELAX(u, v)
    if v.d > u.d + w(u, v):
        v.d = u.d + w(u, v)
        v. = u
        DECREASE-KEY(P, v, v.d) // Update v's key in the priority queue

```

Proof of Correctness (Lemma)

Lemma: Dijkstra's algorithm correctly computes shortest paths from s to all vertices in a graph with non-negative edge weights.

Proof Intuition: Consider the state of the algorithm after growing a tree T where distances to vertices in T are correct. Let u be the closest vertex to s that is not yet in T . The shortest path from s to u must end with an edge $e = (v, u)$, where v is in T (because v is closer to s than u , and u was the closest vertex not in T). This implies that the shortest path to u consists of a shortest path from s to v (which is known to be correct since $v \in T$) followed by the edge (v, u) . When v was processed and added to T , the edge (v, u) would have been relaxed. This relaxation would have updated $u.d$ to its correct shortest path distance from s . Since Dijkstra's algorithm always selects the vertex with the minimum distance estimate outside T , it will eventually select u , and its distance estimate $u.d$ will be the true shortest path distance. By induction, after $n - 1$ steps, the algorithm will have built a shortest path tree containing all reachable vertices.

Implementation and Running Time

The efficiency of Dijkstra's algorithm heavily depends on the implementation of the priority queue.

Priority queue	INSERT	EXTRACT-MIN	DECREASE-KEY	Total Time Complexity ($n = V $, $m = E $)
Array	$O(1)$	$O(n)$	$O(1)$	$O(n^2)$
Binary [[Heaps heap]]	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(m \log n)$

Priority queue	INSERT	EXTRACT-MIN	DECREASE-KEY	Total Time Complexity ($n = V $, $m = E $)
Fibonacci [[Heaps heap]]	$O(1)$ amor- tized	$O(\log n)$ amortized	$O(1)$ amor- tized	$O(m + n \log n)$

The total time complexity is derived from: - n **INSERT** operations (one for each vertex initially). - n **EXTRACT-MIN** operations (one for each vertex added to T).
- At most m **DECREASE-KEY** operations (one for each edge relaxation).

Related

- [[Graphs]]
- [[Shortest Paths]]
- [[Heaps]]